APPLICATION #1: HILBERT CURVE OS ARCHITECTURE

22 Patent Drawings -- ASCII Reference Diagrams

for tracing with Draw.io / Visio / PowerPoint

==

FIG. 1 -- System Architecture Block Diagram

==

```
  +--[ User-space programs ]
  |
  v
+================================================================+
|  100  Hilbert-Addressed Kernel                                 |
|                                                                |
|  +------------------+    +--------------------+                |
|  | 110              |    | 120                |                |
|  | Hilbert          |    | Kernel Subsystem   |                |
|  | Coordinate       |--->| Map                |                |
|  | Engine           |    |                    |                |
|  |                  |    |  (2,0,0) KernEntry |                |
|  |  interleave3()   |    |  (2,1,0) Mem/Heap  |                |
|  |  deinterleave3() |    |  (2,0,1) Scheduler |                |
|  |  distance()      |    |  (2,1,1) IPC       |                |
|  +--------+---------+    |  (2,2,0) FS        |                |
|           |              |  (2,2,1) Drivers   |                |
|           |              +--------+-----------+                |
|           v                       |                            |
|  +------------------+             |                            |
|  | 130              |<------------+                            |
|  | Spatial IPC      |                                          |
|  | Router           |    +--------------------+                |
|  |                  |--->| 150                |                |
|  |  32 pipes        |    | Spatial System     |                |
|  |  16 msg queues   |    | Call Dispatcher    |                |
|  +------------------+    |                    |                |
|                          |  Syscall 0-79      |                |
|  +------------------+    |  Hilbert 80+       |                |
|  | 140              |    +--------------------+                |
|  | Hilbert-Aware    |                                          |
|  | Memory Allocator |                                          |
|  |                  |                                          |
|  |  Sorted free list|                                          |
|  |  Merge sort      |                                          |
|  +------------------+                                          |
+================================================================+
```

```
  |
  v
  +--[ Physical hardware (x86_64) ]
```

FIG. 2 -- 3D Hilbert Coordinate Assignment

==

```
          z (depth)
          ^
          |
          |    (2,2,1) Drivers
          |   /
          |  /   (2,1,1) IPC
          | /   /
          |/   /
    -------+---------> x (functional domain)
         /|
        / |
       /  |   (2,0,0) Kernel Entry
      /   |
     /    |   (2,1,0) Mem/Heap
    /     |
   v      |   (2,0,1) Scheduler
  y (hierarchy)
              (2,2,0) File System

  Legend:
    x-axis = Functional domain (core, mem, IO, ...)
    y-axis = Hierarchy level (low-level, mid, high)
    z-axis = Implementation depth (interface, impl)

  Hilbert curve path (dotted):
    (2,0,0) ---> (2,1,0) ---> (2,0,1) ---> (2,1,1) ---> (2,2,0) ---> (2,2,1)
```

FIG. 3 -- Bit-Interleaving Encoding Flowchart (interleave3)

==

```
    +-------+
    | START |
    +---+---+
        |
        v
  +-------------+
  | r = 0       |
```

```
 | i = 0        |
 +------+------+
        |
        v
   +----+-----+        +--------------------------+
   | i < 10 ? |--Yes-->| bit_x = x mod 2          |
   +----+-----+        | bit_y = y mod 2          |
        |              | bit_z = z mod 2          |
        No             +-------------+------------+
        |                            |
        v                            v
   +---------+         +--------------------------+
   | Return  |         | shift_x = 2^(3*i)        |
   | r       |         | shift_y = 2^(3*i + 1)    |
   | (30-bit |         | shift_z = 2^(3*i + 2)    |
   |  key)   |         +-------------+------------+
   +----+----+                       |
        |                            v
        v              +--------------------------+
     +-----+           | r = r + bit_x * shift_x  |
     | END |           |   + bit_y * shift_y      |
     +-----+           |   + bit_z * shift_z      |
                       +-------------+------------+
                                     |
                                     v
                       +--------------------------+
                       | x = x / 2                |
                       | y = y / 2                |
                       | z = z / 2                |
                       | i = i + 1                |
                       +-------------+------------+
                                     |
                                     +----> (loop back to "i < 10 ?")
```

FIG. 4 -- Reverse Decoding Flowchart (deinterleave3)
==

```
   +-------+
   | START |
   +---+---+
       |
       v
 +--------------+
 | x=0, y=0     |
 | z=0, i=0     |
```

```
 | kk = k      |
 +------+-------+
        |
        v
  +----+-----+       +--------------------------+
  | i < 10 ? |--Yes-->| pow = 2^i                |
  +----+-----+       +-------------+-------------+
        |                          |
        No                         v
        |             +--------------------------+
        v             | x += (kk mod 2) * pow     |
  +-----------+        | kk = kk / 2              |
  | Return    |       +-------------+-------------+
  | [x, y, z] |                      |
  +-----------+                      v
                      +--------------------------+
                      | y += (kk mod 2) * pow     |
                      | kk = kk / 2              |
                      +-------------+-------------+
                                    |
                                    v
                      +--------------------------+
                      | z += (kk mod 2) * pow     |
                      | kk = kk / 2              |
                      | i = i + 1                |
                      +-------------+-------------+
                                    |
                                    +----> (loop back to "i < 10 ?")
```

FIG. 5 -- 2D Hilbert Curve for Page Allocation (8x8)

```
================================================================

   Column: 0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
  Row 0  |  0 |  1 | 14 | 15 | 16 | 19 | 20 | 21 |
        +----+----+----+----+----+----+----+----+
  Row 1  |  3 |  2 | 13 | 12 | 17 | 18 | 23 | 22 |
        +----+----+----+----+----+----+----+----+
  Row 2  |  4 |  7 |  8 | 11 | 30 | 29 | 24 | 25 |
        +----+----+----+----+----+----+----+----+
  Row 3  |  5 |  6 |  9 | 10 | 31 | 28 | 27 | 26 |
        +----+----+----+----+----+----+----+----+
  Row 4  | 58 | 57 | 54 | 53 | 32 | 35 | 36 | 37 |
        +----+----+----+----+----+----+----+----+
  Row 5  | 59 | 56 | 55 | 52 | 33 | 34 | 39 | 38 |
```

```
       +----+----+----+----+----+----+----+----+
 Row 6  | 60 | 61 | 50 | 51 | 46 | 45 | 40 | 41 |
       +----+----+----+----+----+----+----+----+
 Row 7  | 63 | 62 | 49 | 48 | 47 | 44 | 43 | 42 |
       +----+----+----+----+----+----+----+----+

 Arrow: Curve traversal 0 -> 1 -> 2 -> 3 -> 4 -> ... -> 63
 Note: "N must be power of 2"
 Highlighted: nearby 1D indices (e.g., 10,11,12,13) occupy nearby 2D cells
```

FIG. 6 -- IPC Spatial Routing Mechanism
==

```
 +-------------------+                      +-------------------+
 | Sender Subsystem  |                      | Receiver Subsystem|
 | Hilbert key: Ka   |                      | Hilbert key: Kb   |
 +--------+----------+                      +--------+----------+
          |                                          ^
          v                                          |
     +----+------------------------------------------+----+
     |         Hilbert Distance Calculator               |
     |         dist = |Ka - Kb|                          |
     +----+--------------------+-------------------------+
          |                    |
          v                    v
   +--------------+     +--------------+
   | dist < THR ? |     | dist >= THR  |
   +--------------+     +--------------+
          |                    |
          v                    v
   +--------------+     +--------------+
   | Fast Path    |     | Standard     |
   | (shared mem) |     | Pipe         |
   +--------------+     +--------------+

   +-----------------------------------------------+
   | 0x700000  IPC Pipes (32 channels)             |
   | [pipe0][pipe1][pipe2]...[pipe31]              |
   +-----------------------------------------------+
   | 0x730000  Message Queues (16 queues)          |
   | [mq0][mq1][mq2]...[mq15]                      |
   +-----------------------------------------------+
```

FIG. 7 -- System Call Dispatch by Hilbert Layout

==

```
+----------+-------------------+-------------------+
| Syscall  | Functional        | Hilbert Region    |
| Range    | Domain            | Coordinates       |
+----------+-------------------+-------------------+
|  0 - 15  | Process Control   | (2,0,0)-(2,0,3)   |
+----------+-------------------+-------------------+
| 16 - 31  | Memory Management | (2,1,0)-(2,1,3)   |
+----------+-------------------+-------------------+
| 32 - 47  | File System       | (2,2,0)-(2,2,3)   |
+----------+-------------------+-------------------+
| 48 - 63  | Device I/O        | (2,3,0)-(2,3,3)   |
+----------+-------------------+-------------------+
| 64 - 79  | Network           | (2,4,0)-(2,4,3)   |
+----------+-------------------+-------------------+
|   >= 80  | ** Hilbert Ops ** | (2,5,0)+          |
|          | 80=ENCODE         |                   |
|          | 81=DECODE         |                   |
|          | 82=DISTANCE       |                   |
+----------+-------------------+-------------------+
```

FIG. 8 -- Hilbert-Aware Memory Allocator

==

```
+--------------------------------------------+
| Allocation Request                         |
| (from subsystem at Hilbert key K)          |
+---------------------+----------------------+
                      |
                      v
+--------------------------------------------+
| Sorted Free Page List                      |
| (sorted by Hilbert key, merge-sort)        |
|                                            |
| [key=102, pf=0x200] [key=107, pf=0x340]    |
| [key=115, pf=0x500] [key=128, pf=0x680]    |
| [key=142, pf=0x720] [key=156, pf=0x900]    |
+---------------------+----------------------+
                      |
     "Select nearest key to K"
                      |
                      v
+--------------------------------------------+
| Allocated Page                             |
```

```
| (key closest to K -> best locality)        |
+--------------------------------------------+

+--------------------------+
| Merge Sort               |
| O(n log n)               |
| Iterative bottom-up      |
+--------------------------+
```

FIG. 9 -- Boot Chain Architecture

==

```
+------------------+    +----------------------+    +------------------+
| Stage 1 (MBR)    |    | Stage 2              |    | Kernel Entry     |
| @ 0x7C00         | -->| @ 0x8000             | -->| @ 0x100000       |
| 512 bytes        |    |                      |    |                  |
|                  |    | Real -> Protected    |    | Serial init      |
| INT 13h load     |    |   -> Long mode       |    | PIC, PIT, IDT    |
| A20 enable       |    |                      |    | Shell            |
|                  |    | LGDT                 |    |                  |
|                  |    | CR0 PE               |    | Hilbert-addressed|
|                  |    | PAE, IA-32e          |    | subsystem init   |
|                  |    | Paging               |    |                  |
+------------------+    +----------------------+    +------------------+
```

FIG. 10 -- Kernel Memory Map with Hilbert Annotations

==

```
Address Range          Content                   Hilbert
===================    ======================    =========
0x001000 - 0x003FFF    Page Tables (PML4/PDPT/PD)
0x007C00 - 0x007DFF    Stage 1 (MBR)
0x008000 - 0x00BFFF    Stage 2
0x070000               Stack Top (grows down)
                       ~~~~~~~~~~~~~~~~~~~~~~~~
0x100000 - 0x1FFFFF    Kernel Code (1 MB)        (2,0,0)
0x200000 - 0x200FFF    IDT (4 KB)
0x300000 - 0x300017    Tick/Scancode/Key Counters
                       ~~~~~~~~~~~~~~~~~~~~~~~~
0x700000 - 0x720FFF    IPC Pipes (32 channels)   (2,1,1)
0x730000 - 0x770FFF    Msg Queues (16 queues)    (2,1,1)
                       ~~~~~~~~~~~~~~~~~~~~~~~~
0x900000 - 0x901FFF    Module Table (64 slots)
0x902000 - 0x902FFF    Trampoline Table (256)
```

```
0x903000 - 0x942FFF      Module Code Arena (256 KB)

+------------------------------------------------------------------+
|  0x000000                                                    |
|  [Page Tables ]                                              |
|  [MBR       ] 0x7C00                                         |
|  [Stage 2    ] 0x8000                                        |
|  [Stack      ] 0x70000                                       |
|  ~~~~~~~~~~~~~~                                              |
|  [Kernel Code           ] 0x100000  <-- (2,0,0)              |
|  [IDT       ] 0x200000                                       |
|  [Counters   ] 0x300000                                      |
|  ~~~~~~~~~~~~~~                                              |
|  [IPC Pipes             ] 0x700000  <-- (2,1,1)              |
|  [Msg Queues            ] 0x730000  <-- (2,1,1)              |
|  ~~~~~~~~~~~~~~                                              |
|  [Module Table          ] 0x900000                           |
|  [Trampoline            ] 0x902000                           |
|  [Module Arena          ] 0x903000                           |
+------------------------------------------------------------------+
```